

循环结构学习效果检查（学习通）



- 1判断题：当`ecx=0`时，`loop`指令执行循环体的次数为0。
- 2判断题：控制循环是否结束只能安排在循环体之后。
- 3判断题：**LOOP**指令相当于**DEC ECX**和**JNZ**两条指令的功能，但考虑对状态标志的影响，它们是有差别的。
- 4判断题：当`ecx=1`时，执行**LOOP**指令不会发生转移。

汇编语言

11 循环结构与子程序

张青

ieqzhang@zzu.edu.cn

实验五 分支程序设计

1、编写汇编语言程序exp0501.s。编写单分支结构的数字判断程序。在数据段按顺序定义以下三个字符：“N”、“a”、“8”，按照字符顺序进行依次显示并判断是否为数字，如果不是数字0~9，就提示错误“Error! Next One! ”，如果是数字0~9，就提示“Great! ! ! ”。

2、编写汇编语言程序exp0502.s。编写双分支结构的测试高低位程序。

要求：定义双字变量DVAR为57H，测试该数据的最高位，如果为1，则显示字母“L”；如果最高位不为1，则继续测试最低位，如果最低位为1，则显示字母“R”；如果最低位也不为1，则显示字母“M”。

exp0501运行结果展示：

N

Error! Next One!

a

Error! Next One!

8

Great!

- 循环结构

教材章节：5.2

慕课：8

- ▶ 循环指令
- ▶ 计数控制循环
- ▶ 条件控制循环
- ▶ 多重循环

- 子程序结构

- 子程序调用与返回指令

教材章节：5.3.1-5.3.2

慕课：9.2

- ▶ call指令
- ▶ ret指令
- ▶ 子程序的结构
- ▶ 堆栈的作用

学习目标



- 会阅读分析循环结构程序的功能、运行结果
- 会正确使用转移指令实现循环结构
- 会正确使用循环指令实现循环结构
- 熟记call 和ret指令的功能
- 能画出堆栈在子程序调用和返回中的作用
- 能说出子程序调用过程中RIP寄存器的变化
- 会用gdb调试子程序执行过程
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，养成细心、严谨的工作态度
- 养成团队合作，互相帮助的精神

〔例5-14〕简单加密解密程序-加密



.text

#定义代码段

.....

#ECX = 实际输入的字符个数，作为循环的次数

mov ecx, count

xor rbx,rbx

#RBX清零，指向源字符串中的每个字符

mov al,key[rip]

#AL=密钥

lea rsi,msg1[rip]

encrypt:

xor [rsi+rbx],al

#异或加密

inc rbx

#指向下一个字符

loop encrypt

#相当于



dec ecx
jnz encrypt

lea rax, msg2[rip]

call dispmsg

lea rax,msg1[rip]

#显示加密后的密文

call dispmsg

〔例5-14〕简单加密解密程序-解密



#ECX=实际输入的字符个数，作为循环的次数

```
mov ecx, count
```

```
xor rbx,rbx
```

#RBX清零，指向加密后字符串中的每个字符

```
mov al,key[rip]
```

#AL=密钥

```
decrypt:xor [rsi+rbx],al
```

#异或解密

```
inc rbx
```

```
loop decrypt
```

#相当于



```
dec ecx  
jnz decrypt
```

```
lea rax, msg3[rip]
```

```
call dispmsg
```

```
lea rax, msg1[rip]
```

#显示解密后的明文

```
call dispmsg
```

〔例5-14〕 优化



采用反向访问（从最后一个元素到第一个元素）的顺序，可以直接利用rcx作为字符位置索引，可优化汇编指令序列

```
main:                                # 程序执行起始位置
    #ECX = 实际输入的字符个数，作为循环的次数
    mov ecx, count
    #xor rbx,rbx → 删去
    mov al, key[rip]                #AL = 密钥
    lea rsi,msg1[rip]
encrypt: xor -1[rsi+rcx],al          #异或加密
    #inc rbx → 删去
    loop encrypt                    #处理下一个字符
```


〔例5-15〕 查找程序



找出数据段中以string开始的10个字节数串中的第一个非零项，将其数值所在位置显示出来；若没有找到，显示 “not found!”。

```
.data #定义数据段  
str: .byte 0,0,-1,0,0,0,39,67,9,10  
.equ count, .- str  
msg: .ascii " not found!\n"
```

〔例5-15〕 查找程序-代码段



```
    lea rbx,str[rip]
    mov ecx,count                #计数器ecx=字符串长度
    mov eax, -1                 #eax记录下标
again: inc eax
    cmp byte ptr [rbx+rax], 0    #是否为0
    loopz again                 #ZF=1,即是0则继续比较
    jecxz nofound               #循环结束, 如果ecx=0, 表示没有找到
    call dispsid                #显示下标
    jmp done
nofound:
    lea rax, msg[rip]
    call dispmsg                #显示not found
done:
```

如果查找数串中出现的第一个0, 将loopz改为loopnz。

- 实际的应用问题
 - 单纯的分支或循环结构
 - 循环体中具有分支结构
 - 分支体中采用循环结构
 - 循环体中嵌套有循环，即形成多重循环结构
- 如果内外循环之间没有关系
 - 比较容易处理
- 如果需要传递参数或利用相同的数据
 - 问题比较复杂

冒泡法的排序过程



比较遍数

数据	1	2	3	4
587	-632	-632	-632	-632
-632	587	234	-34	-34
777	234	-34	234	234
234	-34	587	587	587
-34	777	777	777	777

从小到大排序

冒泡法的排序过程

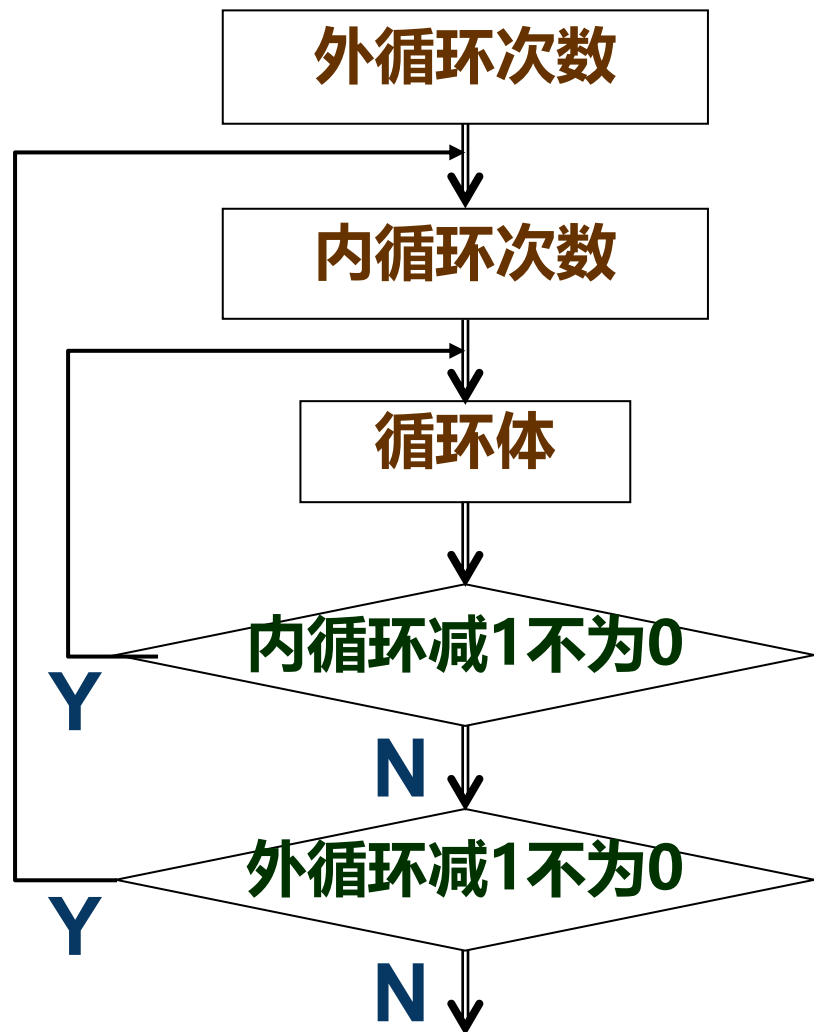


比较轮次

数据	1	2	3	4
587	-632	-632	-632	-632
-632	587	234	-34	-34
777	234	-34	234	234
234	-34	587	587	587
-34	777	777	777	777

从小到大排序

〔例5-16〕冒泡法排序程序



外循环次数 $i = N - 1$
第 j 轮内循环次数 $= N - j$

C函数bubble_sort



```
void bubble_sort(int a[],int n){  
    for(int i=1;i<=n-1;i++)  
        for(int j=0; j<n-i;j++)  
            if(a[j]>a[j+1]) {  
                int t =a[j];  
                a[j]=a[j+1];  
                a[j+1]=t;  
            }  
}
```

编译生成的汇编语言-1



```
bubble_sort:      #rcx=a, edx=n
    mov    r11d, 1  #外循环次数 r11d=i=1
    jmp    .L2      #外循环
.L3:              #内循环
    add     eax, 1   #j++
.L5:      mov    r8d, edx  # r8d = edx = n
    sub     r8d, r11d    # 内循环次数 r8d = n-i
    cmp     r8d, eax     # 内循环条件 ,n-i ≤ j ?
    jle     .L7          # 若n-i ≤ j ,则内循环结束
    movsx   r8, eax      # 内循环体, r8=j
    lea     r10, [rcx+r8*4]  #r10=a[j]的地址
    mov     r9d, DWORD PTR [r10]  # r9d =a[j]的值
    mov     r8d, DWORD PTR 4[rcx+r8*4]
    #4[rcx+r8*4]=a[j+1],r8d=a[j+1]的值
    cmp     r9d, r8d     #a[j]≤a[j+1]?
    jle     .L3
```


编译生成的汇编语言-2



```
    mov    DWORD PTR [r10], r8d    # a[j+1]送到a[j]
    lea     r8d, [r11+rax]          # r8d = j+1
    movsxd r8, r8d
    mov     DWORD PTR [rcx+r8*4], r9d # a[j]送到a[j+1]
    jmp     .L3                     #下一轮内循环
.L7: add     r11d, 1                # i++,外循环次数加1
.L2:                                     #外循环条件判断
    cmp     r11d, edx              # 比较i与n,edx = n
    jge     .L8                    # 若i≥n,退出
    mov     eax, 0                 # eax = j = 0, 内循环初始化
    jmp     .L5                    #进入内循环
.L8:                                     #循环结束
    ret
```

优化: eg0516a.S



```
lea    r10, [rcx+r8*4]
mov     r9d, DWORD PTR [r10]
mov     r8d, DWORD PTR 4[rcx+r8*4]
cmp     r9d, r8d
jle     .L3
mov     DWORD PTR [r10], r8d
lea     r8d, [r11+rax]
movsx   r8, r8d
mov     DWORD PTR [rcx+r8*4], r9d
```

#r10=a[j]的地址
r9d =a[j]的值
#4[rcx+r8*4]=a[j+1],r8d=a[j+1]的值
#a[j]>a[j+1]?

a[j+1]送到a[j]
r8d = j+1

a[j]送到a[j+1]

lea r10, [rcx+r8*4]
mov r9d, DWORD PTR [r10]
mov r8d, DWORD PTR 4[r10]
cmp r9d, r8d
jle .L3
mov DWORD PTR [r10], r8d
mov DWORD PTR 4[r10], r9d

优化: eg0516b.S



```
.L3:  sub    eax, 1          #内循环计数-1
      add    r10, 4        #移到下一个元素
.L5:  cmp    eax, 0        #内循环计数是否为0
      jle    .L7          #若内循环计数≤0, 则内循环结束
      mov    r9d, DWORD PTR [r10]
      mov    r8d, DWORD PTR 4[r10]
      cmp    r9d, r8d
      jle    .L3
      mov    DWORD PTR [r10], r8d  #a[j]=a[j+1]
      mov    DWORD PTR 4[r10], r9d #a[j+1]=a[j]
      jmp    .L3
.L7:  add    r11d, 1        #i++,外循环次数加1
.L2:  cmp    r11d, edx      #判断外循环条件, 比较i与n
      jge    .L8          #若i≥n, 则退出外循环
      mov    r10, rcx      #内循环初始化, r10 = rcx = &a[0],
      mov    eax, edx      #eax=edx=n
      sub    eax, r11d     #eax = n-i
      jmp    .L5
```

LOOP指令实现: eg0516c.S



	<code>mov r12, rcx</code>	<code>#r12 = rcx = &a[0], 等同于 lea r12, [rcx]</code>
	<code>mov ecx, edx</code>	<code>#ecx = edx = n</code>
	<code>sub ecx, 1</code>	<code>#外循环次数 ecx = n - 1</code>
	<code>jmp .L2</code>	
<code>.L3:</code>	<code>sub eax, 1</code>	
	<code>add r10, 4</code>	
<code>.L5:</code>	<code>cmp eax, 0</code>	<code>#内循环计数是否为0</code>
	<code>jle .L7</code>	
	<code>.....</code>	
	<code>cmp r9d, r8d</code>	<code>#比较a[i]和a[i+1]</code>
	<code>jle .L3</code>	
	<code>.....</code>	<code>#交换数据</code>
	<code>jmp .L3</code>	
<code>.L7:</code>	<code>loop .L2</code>	<code>#外循环控制</code>
<code>.L8:</code>	<code>ret</code>	
<code>.L2:</code>	<code>mov eax, ecx</code>	<code>#eax = 内循环次数 = 外循环计数</code>
	<code>mov r10, r12</code>	<code>#r10 = r12 = &a[0]</code>
	<code>jmp .L5</code>	<code># 开始内循环</code>

ecx=0时, 会出现死循环。

〔例5-17〕 字符剔除程序



现有一个以0结尾的字符串，要求剔除其中的空格字符。

```
.data #定义数据段  
    string: .ascii "This is a test"  
    .byte 0
```

〔例5-17〕 字符剔除程序



```
        lea rsi,string[rip]
outlp:  cmp byte ptr[rsi],0
        jz done
again:  cmp byte ptr [rsi],' '
        jnz next
        mov rdi,rsi
inlp:   inc rdi
        mov al,[rdi]
        mov -1[rdi],al
        cmp byte ptr [rdi],0
        jnz inlp
        jmp again
next:   inc rsi
        jmp outlp
done:
```

#外循环，先判断后循环

#为0结束

#是否是空格

#不是空格继续循环

#是空格，剔除分支

#该分支是循环程序

#前移一个位置

#内循环，先循环后判断

#内循环结束处

#再次判断是否为空格（处理连续空格）

#继续对后续字符进行判断处理

#外循环结束处

字符剔除优化



```
void del_char(char *s, char t){
    char *p1 = s, *p2=s;
    while (*p1!= '\0') {
        if (*p1 != t) {
            *p2++ = *p1;
        }
        p1++;
    }
    *p2= '\0';
}
```

```
del_char:# rcx=p1=s, r8=p2=s, dl=空格
        mov r8, rcx
        jmp .L7
.L8: add rcx, 1          # p1+1, 下一个字符
.L7: movzx eax, BYTE PTR [rcx] # al = *p1
        test al, al      # 是否为0
        je .L10         # 是0, 则结束
        cmp al, dl      # 是否是要删除的字符, dl=t
        je .L8          # 是, 转到.L8
        mov BYTE PTR [r8], al # 不是要删除的字符
        lea r8, 1[r8]      # p2+1
        jmp .L8          # 下一个字符
.L10:
        mov BYTE PTR [r8], 0
        ret
```



• 子程序

子程序结构



- 经常用到的应用问题编写成一个通用子程序
- 大型处理过程分解成能够解决的模块
- 使用子程序可以
 - 使程序的结构更为清楚
 - 程序的维护更为方便
 - 有利于大程序开发时的多个程序员分工合作

子程序 (Subroutine)
= 函数 (Function)
= 过程 (Procedure)

〔例5-18〕子程序结构-C语言源程序



```
#include <stdio.h>
int mult2(int x, int y){
    int t = x*y;
    return t;
}
void multstore(int x, int y, int *dest) {
    int t = mult2(x, y);
    *dest = t;
}
int main(){
    int m;
    multstore(3,5,&m);
    printf("3 * 5 =%d",m);
    return 0;
}
```

〔例5-18〕 mult2汇编语言程序



```
int mult2(int x, int y){  
    int t = x*y;  
    return t;  
}
```

```
mult2: #Windows,入口参数ecx:x, edx:y  
    mov    eax, ecx  
    imul   eax, edx    # eax=x*y  
    ret                    #函数返回, 返回值eax
```

```
mult2: #Linux,入口参数edi:x, esi:y  
    mov    eax, edi  
    imul   eax, esi    #eax=x*y  
    ret                    #函数返回, 出口参数eax
```

编译命令:

```
gcc -S -Og -masm=intel -o eg0518.s eg0518.c
```

〔例5-18〕 multstore汇编语言程序



```
void multstore(int x, int y, int *dest) {  
    int t = mult2(x, y);  
    *dest = t;  
}
```

- Linux系统入口参数
 - edi:x
 - esi:y
 - edx:dest

multstore: #Windows, 入口参数 ecx:x, edx:y, r8:dest

push	rbx	#保护rbx
sub	rsp, 32	#建立栈帧
mov	rbx, r8	#保存r8, 防止mult2改变r8
call	mult2	# 调用函数mult2
mov	DWORD PTR [rbx], eax	
add	rsp, 32	#平衡堆栈
pop	rbx	#恢复rbx
ret		#函数返回, 出口参数eax

汇编子程序结构



.text

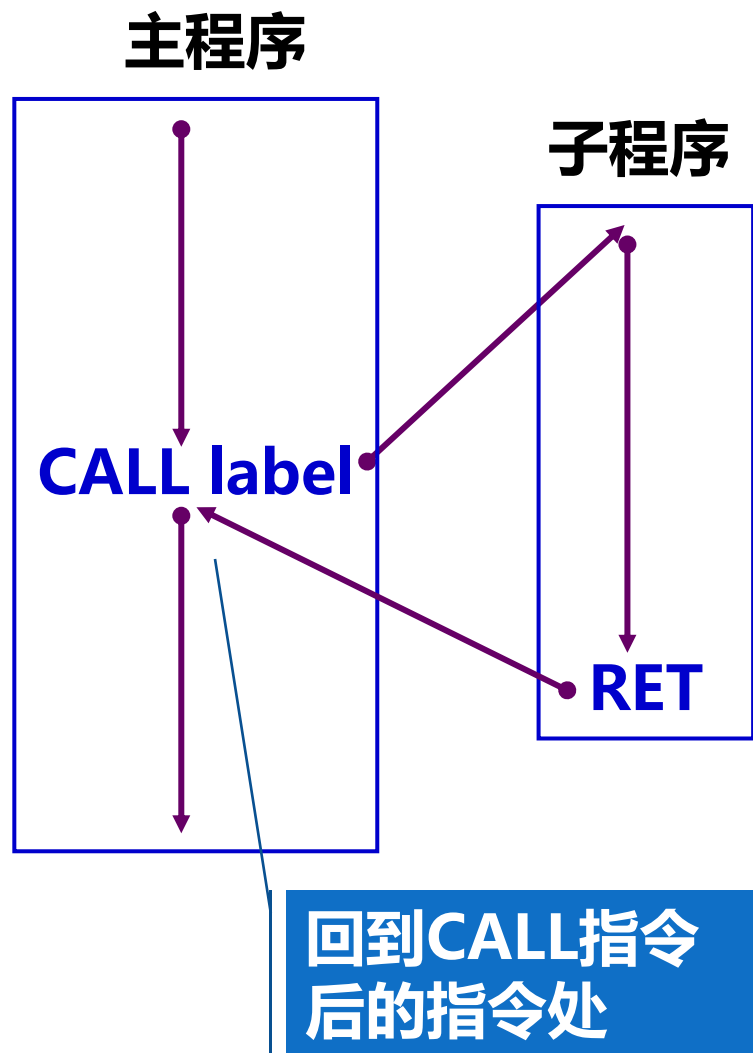
子程序名: #子程序开始
#子程序体
ret #子程序返回

main: #程序入口
#主程序结束

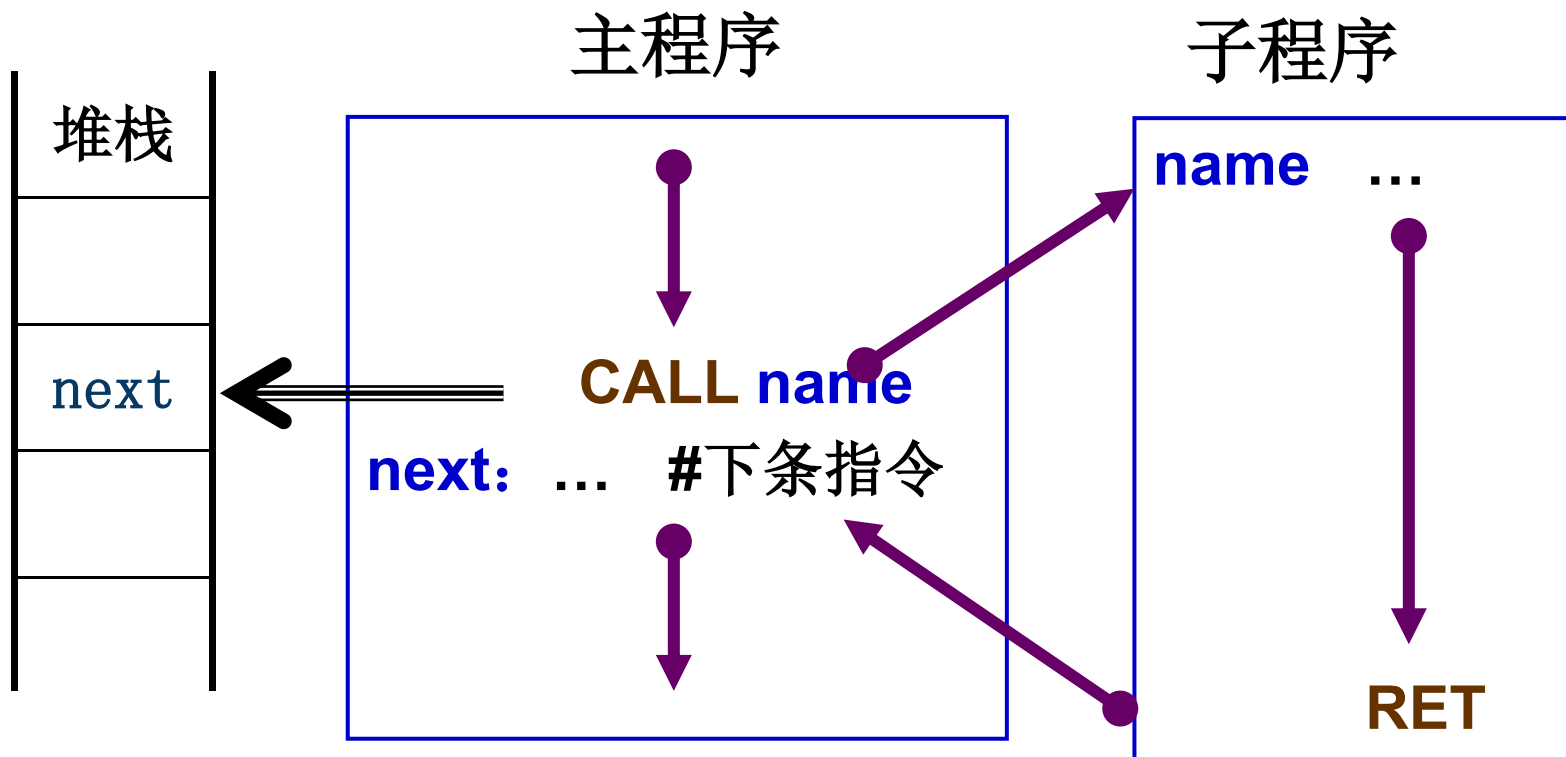
子程序名: #子程序开始
#子程序体
ret #子程序返回

子程序调用和返回指令

- 子程序
 - 与主程序分开、完成特定功能的一段程序
- 主程序(调用程序)
 - 执行调用指令CALL调用子程序
- 子程序(被调用程序)
 - 执行返回指令RET返回主程序



CALL的调用和RET的返回



思考如下代码片段的功能

```
call next
next:  pop rax    #RAX= ?
```

1. 子程序调用指令CALL

- CALL指令用在主程序中，实现子程序的调用
 - 分成段内调用（近调用）和段间调用（远调用）
 - 64位模式下，目标地址采用相对寻址或间接寻址
 - 32位兼容模式下，目标地址可以采用相对寻址、间接寻址、或直接寻址
- CALL指令的两个功能：
 - 入栈返回地址：将CALL下条指令的地址压入堆栈
 - 目标地址送RIP $RSP \leftarrow RSP - 8, SS:[RSP] \leftarrow RIP, RIP \leftarrow \text{目标地址}$
 - 实模式：16位，32位保护模式：32位

64位模式CALL指令格式

CALL label

#相对近调用

#调用标号指定的子程序

#相对寻址

CALL reg64/m64

#绝对近调用

#调用寄存器或存储器指定地址的子程序

#间接寻址

CALL m16:16/m16:32/m16:64 #远调用

#调用寄存器或存储器指定地址的子程序

#间接寻址

子程序与call指令在同一代码段

子程序与call指令不在同一代码段

相对近调用



.text #eg0518.lst

.....

mult2:

14 0000 89C8 mov eax, ecx

.....

multstore:

目标地址=0x0013+0xFFFFFFFFED=0x0000

.....

28 000e E8EDFFFF call mult2

28 FF

29 0013 8903 mov DWORD PTR [rbx], eax

.....